

HIGH-LEVEL DESIGN (HLD)

RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

1. System Overview

1.1 Problem Definition

Traditional customer support systems suffer from slow response times, high operational costs, and inability to leverage organizational knowledge effectively. Support agents spend significant time searching through documentation and knowledge bases to answer repetitive queries, leading to customer dissatisfaction and agent burnout.

This project solves these challenges by building a Retrieval-Augmented Generation (RAG)-based Customer Support Assistant that:

- Processes and indexes a company's PDF-based knowledge base into a searchable vector store
- Retrieves the most contextually relevant information when a user submits a query
- Generates accurate, grounded responses using a Large Language Model (LLM)
- Orchestrates the entire pipeline using a graph-based workflow (LangGraph)
- Escalates complex or uncertain cases to human agents through a HITL mechanism

1.2 Scope of the System

The system is scoped to handle:

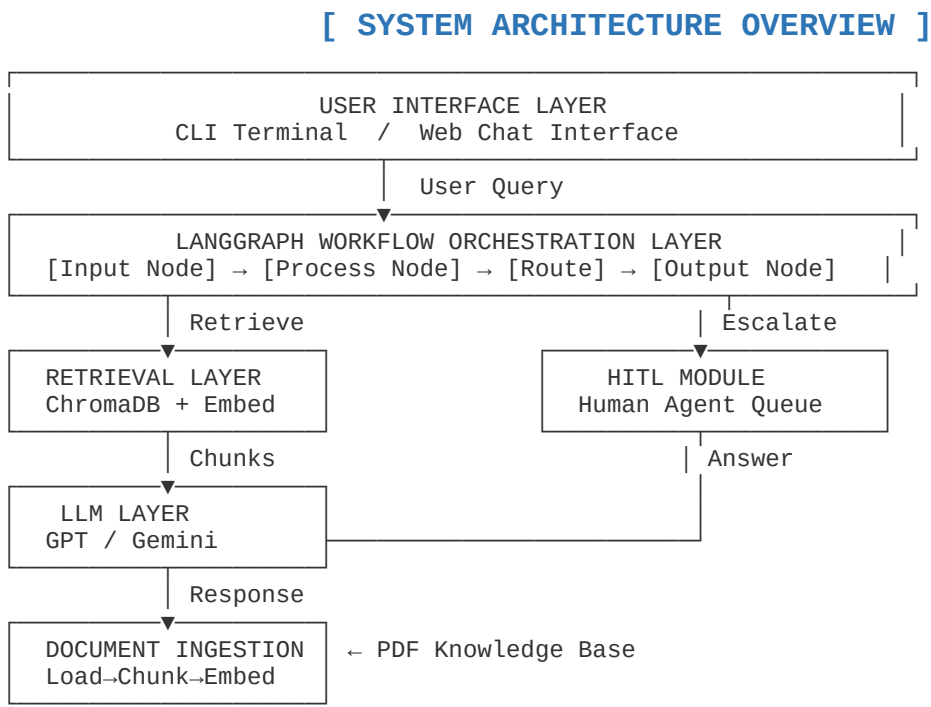
- Single or multiple PDF knowledge bases as the primary data source
- Natural language queries from end users via CLI or web interface
- Intent classification and conditional routing (auto-answer vs. escalate)
- Human agent integration for escalated queries
- Stateful workflow execution using LangGraph

Out of scope (future enhancements):

- Real-time document ingestion / live data feeds
- Multi-modal inputs (images, audio)
- Multi-language support

2. Architecture Diagram

The system is organized into six major layers, each responsible for a distinct function:



3. Component Description

3.1 Document Loader

Responsible for ingesting raw PDF files from the file system. Uses LangChain's PyPDFLoader (or UnstructuredPDFLoader for complex layouts) to parse text content page-by-page. Handles encoding normalization, whitespace cleanup, and metadata extraction (file name, page number, creation date) which are stored alongside each chunk for traceability.

3.2 Chunking Strategy

Implements Recursive Character Text Splitter with the following parameters:

- Chunk size: 800 tokens — large enough for context, small enough for precise retrieval
- Chunk overlap: 100 tokens — maintains semantic continuity across chunks
- Splitter hierarchy: paragraph → sentence → word — preserves natural language boundaries

Each chunk is enriched with metadata: source file, page number, and chunk index.

3.3 Embedding Model

Uses sentence-transformers (all-MiniLM-L6-v2) for local, cost-free embedding generation. Each chunk is converted to a 384-dimensional dense vector. For production, OpenAI text-embedding-3-small can be used for higher quality at cost. Embeddings capture semantic meaning, enabling similarity-based retrieval.

3.4 Vector Store (ChromaDB)

ChromaDB is chosen as the vector database for its zero-configuration local setup, persistent storage, and native Python integration. Embeddings are stored with associated metadata enabling filtered retrieval. The collection is persisted to disk, avoiding re-embedding on each startup.

3.5 Retriever

Implements cosine similarity search to retrieve the top-k (default k=5) most relevant chunks for each query. Applies a similarity score threshold (0.35) to filter low-relevance chunks. Returns an ordered list of Document objects with content and metadata.

3.6 LLM (Large Language Model)

Uses OpenAI GPT-4o-mini (or Google Gemini 1.5 Flash as fallback) as the generation backbone. The LLM receives a structured prompt containing: (1) system role definition, (2) retrieved context chunks, and (3) the user query. Constrained to only use provided context — no hallucination permitted.

3.7 Graph Workflow Engine (LangGraph)

LangGraph provides a DAG (Directed Acyclic Graph) based workflow runtime. The graph maintains a shared State object that flows between nodes. Supports conditional edges enabling dynamic routing. The graph is compiled once and reused across queries for efficiency.

3.8 Routing Layer

Analyzes LLM output confidence and context relevance to decide routing:

- HIGH confidence + RELEVANT chunks → direct answer to user
- LOW confidence OR no relevant chunks → escalate to HITL
- AMBIGUOUS query → request clarification from user

3.9 HITL Module

When escalation is triggered, the system pauses the graph using LangGraph's interrupt mechanism. The query, context, and partial answer are forwarded to a human agent queue. The human agent reviews and provides a verified response, which is injected back into the graph to complete the workflow.

4. Data Flow

4.1 Document Ingestion Flow (One-time Setup)

1. PDF file(s) are loaded from the /data directory using PyPDFLoader
2. Raw text is extracted page-by-page with source metadata preserved
3. Text is split into overlapping chunks using RecursiveCharacterTextSplitter
4. Each chunk is embedded using the configured embedding model
5. Embedding vectors + metadata are persisted to ChromaDB collection

4.2 Query Lifecycle (Runtime)

6. User submits a natural language query via CLI or web interface
7. LangGraph initializes a new State object with the query
8. Input Node validates and preprocesses the query
9. Process Node embeds the query and retrieves top-k chunks from ChromaDB
10. LLM generates a response using retrieved chunks as context
11. Router evaluates confidence score and context relevance
12. If HIGH confidence: Output Node formats and returns response to user
13. If LOW confidence: HITL Node pauses graph, notifies human agent
14. Human agent provides verified response; graph resumes and delivers answer

5. Technology Choices

5.1 Why ChromaDB

- Zero-setup: runs embedded in Python process, no server required
- Persistent storage: data survives restarts without re-embedding
- Native metadata filtering: enables source-aware retrieval
- LangChain integration: plug-and-play with existing pipeline
- Open-source: no vendor lock-in, free for local use

5.2 Why LangGraph

- Stateful execution: shared State object eliminates repeated data passing
- Conditional edges: enables dynamic routing based on runtime conditions
- HITL support: built-in interrupt/resume for human intervention
- Debuggability: graph visualization and step-by-step tracing
- Composability: nodes are pure Python functions, easy to test

5.3 LLM Choice

Model	Provider	Strengths	Use Case
GPT-4o-mini	OpenAI	Cost-effective, fast	Primary (Production)
Gemini 1.5 Flash	Google	Long context, free tier	Fallback / Dev
Llama 3 (local)	Meta/Ollama	Privacy, zero cost	Air-gapped environments

5.4 Additional Tools

Tool	Purpose
LangChain	Document loading, splitting, retrieval orchestration
sentence-transformers	Local embedding generation (no API cost)
PyPDF2 / pdfplumber	PDF text extraction
FastAPI	REST API layer for web interface
Python-dotenv	Environment variable management

6. Scalability Considerations

6.1 Handling Large Documents

- Batch embedding: process chunks in batches of 100 to avoid memory overflow
- Async ingestion: use asyncio for parallel PDF loading
- Chunking optimization: adaptive chunk size based on document structure
- Migration path: replace ChromaDB with Pinecone or Weaviate for 10M+ vectors

6.2 Increasing Query Load

- LangGraph graph is stateless per query — horizontally scalable
- FastAPI + Uvicorn supports async concurrent request handling
- Retriever can be cached with Redis for repeated queries
- LLM calls can be batched or streamed to reduce perceived latency

6.3 Latency Concerns

Component	Typical Latency	Optimization
Embedding (local)	50-100ms	GPU acceleration, batch processing
ChromaDB retrieval	10-30ms	Index pre-warming, connection pooling
LLM generation	500-2000ms	Streaming responses, smaller models
Total pipeline	600-2200ms	Async execution, caching

Target SLA: P95 response time under 3 seconds for auto-answered queries.